

CUADERNO DE DIVULGACIÓN · DEEP RL · EDICIÓN 2

Arkanoid DRL Learning Lab

Cinco algoritmos de aprendizaje por refuerzo aprendiendo a **pasar niveles que nunca han visto**. Esta segunda edición da la vuelta a la tesis: el cuello de botella nunca fue el algoritmo ni «entrenar más», sino **cómo planteábamos la tarea**. Al arreglar el reloj, la recompensa, la observación y la evaluación, varios algoritmos aprendieron a apuntar y generalizar. Con protocolo **congelado** y **multi-semilla**: cada cifra tiene su artefacto.

PPO

DQN

SAC

World Model

World Model RNN

PyTorch · MPS

Protocolo congelado

5 semillas

AUTOR

Roberto Canales Mora · con Claude

WEB

www.robertocanales.com

REPOSITORIO

github.com/rcanalescoder/DRLArkanoid

PROTOCOLO

`frozen_hash a1ab7ce18d7bad6b`

El problema no era el algoritmo ni el tiempo; era la formulación.

Edición 2 · resultados sobre niveles no vistos

Pasar niveles nunca vistos

Qué queremos de verdad, por qué es difícil y cómo se mide.

De qué va este documento y **por qué te importa**: vas a ver a varios agentes aprender, desde cero y sin que nadie les explique las reglas, a **limpiar niveles de Arkanoid que no estaban en su entrenamiento**. Y vas a ver, con datos honestos, qué de verdad desbloqueó el problema. Spoiler: no fue el algoritmo.

El objetivo no es que el agente «sobreviva» moviendo la pala, sino que **resuelva** el nivel: que rompa todos los ladrillos. Y no en un nivel que ha practicado mil veces, sino en niveles **nuevos**, con los ladrillos colocados de formas que no ha visto. Eso es lo difícil, y es lo que de verdad mide si ha **aprendido a jugar** en lugar de **memorizar**.

La regla de oro de este documento

La métrica de cabecera: éxito en niveles de TEST no vistos

A lo largo de todo el documento, la cifra que importa es el **porcentaje de niveles NUEVOS que el agente limpia por completo**, jugando «a lo seguro» (sin explorar). Nunca la recompensa de entrenamiento: un agente puede acumular mucha recompensa y aun así no pasar ni un nivel. Premio $\neq$ logro.



La analogía del examen

Estudiar con los exámenes de otros años está bien, pero el examen de verdad trae preguntas nuevas. Si solo sabes repetir los ejercicios vistos, suspendes. Medir en niveles **no vistos** es traer preguntas nuevas: separa al que entendió del que memorizó.

Para que «niveles nuevos» signifique algo, separamos desde el principio los niveles en dos bolsas que **no se tocan**: una para **entrenar** y otra, oculta, para **examinar**. El agente nunca entrena con los niveles del examen. Y medimos sobre tres exámenes distintos, de menos a más exigente:



TEST-ID (500 niveles)

Mismo «estilo» que el entrenamiento (mismas familias de patrones) pero niveles concretos no vistos. Mide si generaliza dentro de lo conocido.



TEST-OOD-patrón (250)

Familias **estructuralmente nuevas** (túneles, diagonales, anillos, huecos centrales...). Mide si generaliza a formas que nunca vio.



TEST-OOD-dificultad (199)

Niveles más **densos** (casi llenos). Mide si aguanta cuando hay muchísimos ladrillos.

Todos los niveles de examen están **verificados como limpiables** por un oráculo determinista: ninguno es imposible. Si el agente no lo limpia, es cosa suya, no del nivel.

Quédate con esto

Queremos agentes que **pasen niveles nuevos**, no que sobrevivan. Lo medimos con el **% de niveles no vistos que limpian**, jugando a lo seguro. Todo lo demás del documento gira alrededor de esa cifra.

El agente ciego

Sobrevive, pero no limpia. Y ahí empieza el misterio.

El primer agente solo veía **seis números**: la posición y la velocidad de la bola, y la posición de la pala. Nada más. En particular, **no veía los ladrillos**. Con esa información puede aprender a poner la pala debajo de la bola y devolverla... pero no puede saber **hacia dónde** conviene dirigirla, porque no sabe dónde quedan ladrillos por romper.

Ojo con esta palabra · observación. En RL, la **observación** (u **estado**) es «lo que el agente ve» en cada instante. Si la observación no contiene una información, el agente **no puede** usarla, por muy listo que sea el algoritmo. El agente ciego tenía una observación **parcial**: le faltaba lo esencial, los ladrillos.

0%

NIVELES DISPERSOS
LIMPIADOS

~2/28

LADRILLOS QUE ROMPÍA

≈56%

«ÉXITO» EN REJILLA LLENA*

* Ese 56% es la trampa, no un logro: en una rejilla **llena**, una bola que rebota durante ~2.200 pasos acaba tocando casi todo por pura física. Es **supervivencia degenerada**, no inteligencia. En cuanto los ladrillos están **dispersos**, hay que **apuntar** — y ahí el ciego se queda en 0%.



El misterio

El agente entrena y entrena, la recompensa sube... y sigue sin pasar niveles dispersos. La tentación fácil es decir «necesita entrenar más». **Es falso**, y demostrarlo es el corazón de este documento.



La pista

Un agente que no ve los ladrillos no puede apuntar. Su techo no es de entrenamiento: es de **información**. Pero, como veremos, la observación no era el único muro — había tres, y ninguno era el algoritmo.

Quédate con esto

El agente ciego **sobrevive pero no limpia** (0% en niveles dispersos). No es que el algoritmo sea malo ni que falte entrenamiento: es que **le falta información y la tarea estaba mal planteada**. Vamos a diagnosticarlo.

Tres muros, ninguno el algoritmo

El reloj, la recompensa y la observación. Uno por uno.

Aquí está el giro de la historia. El PDF anterior concluía que «el cuello de botella es el tiempo de entrenamiento». **Los datos lo refutan.** El cuello de botella era la **formulación** de la tarea, en tres capas que conviene separar porque se arreglan por separado.

I Muro 1 · El reloj: ganar era imposible

Cada partida (un **episodio**) tenía un límite de pasos. Si se agotaba, la partida terminaba «en tablas». El problema: limpiar todos los ladrillos exige **muchos** pasos, y el límite estaba puesto demasiado corto. Ganar no era difícil: era **literalmente imposible**, porque no daba tiempo material.

Ojo con esta palabra · límite de episodio (timeout). Es el número máximo de pasos que dura una partida. Si limpiar el nivel necesita más pasos que el límite, el agente **no puede ganar aunque juegue perfecto**. Estábamos midiendo contra un reloj imposible.

El cálculo que lo delata

Romper 28 ladrillos exige del orden de **~1.760 pasos** (hay que ir, golpear, volver...). El límite estaba en **600**. Un rastreador **perfecto**, sin red neuronal, pasaba del **0%** al **37,6%** solo con arreglar el reloj (timeout proporcional al nº de ladrillos). El algoritmo no había cambiado ni un bit.

I Muro 2 · La recompensa: premiábamos lo que no queríamos

Para «ayudar» al agente le dábamos una recompensa extra, Φ (shaping), por acercar la pala a la bola. La intención era buena; el efecto, un sabotaje. Esa recompensa **arreaba** al agente hacia un rebote casi vertical y una estrategia **degenerada**: mantener la bola arriba sin importar dónde, limpiando como mucho una columna. Optimizaba el premio... alejándose del objetivo.

Ojo con esta palabra · reward shaping. Es añadir recompensas auxiliares para guiar el aprendizaje. **Bien hecho** acelera sin cambiar la solución óptima. **Mal hecho** crea una solución óptima **distinta** de la que querías: el agente aprende a maximizar tu pista, no tu meta. Aquí Φ fue lo segundo. (Y, como veremos en los datos, quitarlo **mejora** el resultado.)

Corrección de rumbo respecto al PDF anterior: allí se presentaba Φ como algo bueno («acelera sin cambiar la política óptima»). En la práctica era **el saboteador**. Lo decimos claro.

I Muro 3 · La observación: ciego a los ladrillos (el techo real)

Arreglado el reloj y la exploración, el ciego seguía sin apuntar: ganaba por **supervivencia** en niveles llenos ($\approx 56\%$) pero **0%** en dispersos. Este es el muro de verdad: **sin ver los ladrillos no se puede apuntar ni generalizar**. Ninguna cantidad de entrenamiento arregla una observación incompleta.

► PARA CURIOSOS · OBSERVABILIDAD PARCIAL

Cuando la observación no contiene todo lo relevante del estado real, el problema deja de ser un MDP limpio y se vuelve **parcialmente observable**. El agente toma decisiones con información insuficiente; el techo de rendimiento lo pone la observación, no el algoritmo. La solución no es un algoritmo más potente: es **darle de ver** lo que necesita.

El diagnóstico, en una frase

No era el algoritmo ni «entrenar más». Eran tres cosas de la **formulación**: un **reloj** imposible, una **recompensa** que premiaba lo que no queríamos, y una **observación** ciega a los ladrillos. Arreglémoslas.

Darle un reloj justo, una meta limpia y ojos

Timeout proporcional + recompensa corregida + visión + currículum + niveles procedurales.

La solución no fue un algoritmo nuevo, sino **replantear la tarea**. Cinco cambios, cada uno atacando una causa concreta:

- 1 · Reloj justo** Timeout **proporcional al nº de ladrillos** (no un número fijo corto): ganar pasa a ser posible.
- 2 · Meta limpia** Quitar el shaping Φ que arreaba a sobrevivir; la recompensa premia romper ladrillos y ganar, no «estar cerca de la bola».
- 3 · Ojos** Añadir a la observación la **matriz de ocupación** de ladrillos (8×10): el agente por fin **ve** qué queda por romper.
- 4 · Encoder conv** Una pequeña red **convolucional** que lee esa matriz como lo que es: una rejilla con estructura espacial.
- 5 · Currículo** Empezar por niveles fáciles (pocos ladrillos) e ir subiendo, sobre **niveles generados al azar** con familias de patrones, y examinar en una bolsa **aparte**.

La pieza «ojos»: una matriz de ocupación, NO píxeles

Es importante ser honestos con qué ve el agente ahora. **No** le damos la imagen del juego ni le explicamos las reglas. Le damos una **matriz 8×10** que dice, por cada celda, si hay ladrillo o no (1/0), más los seis números de antes (bola y pala). Es una representación **estructurada** y ligera, no percepción visual desde píxeles como en el Atari clásico.

Ojo con esta palabra · convolución. Una capa convolucional es un pequeño «detector de patrones» que se desliza por la rejilla mirando vecindarios locales. Aquí actúa como un **encoder ligero sobre una matriz de ocupación**: aprovecha que dos ladrillos vecinos están relacionados. No es «visión por píxeles»; es leer una rejilla con estructura.

► PARA CURIOSOS · ¿POR QUÉ CONV Y NO UNA LISTA PLANA?

Se podría aplanar la matriz a una lista de 80 números y meterla en una red normal (un **MLP**).

Funciona... peor. La convolución incorpora el **sesgo espacial** (lo cercano se relaciona), aprende más rápido y generaliza mejor a configuraciones nuevas. Lo cuantificamos en el Capítulo 5 (ablación): quitar la conv cuesta ~20 puntos de éxito.

■ Currículo y niveles procedurales: aprender a generalizar

En vez de un único nivel fijo, generamos **cientos** de niveles con familias de patrones (dispersión, filas, columnas, bloques, simétricos) y los repartimos en **train** y **test** disjuntos. El agente entrena primero con los fáciles (pocos ladrillos) y va desbloqueando los difíciles a medida que mejora — un **currículo**, como un temario que sube de nivel. Así no memoriza un nivel: aprende una **política general**.

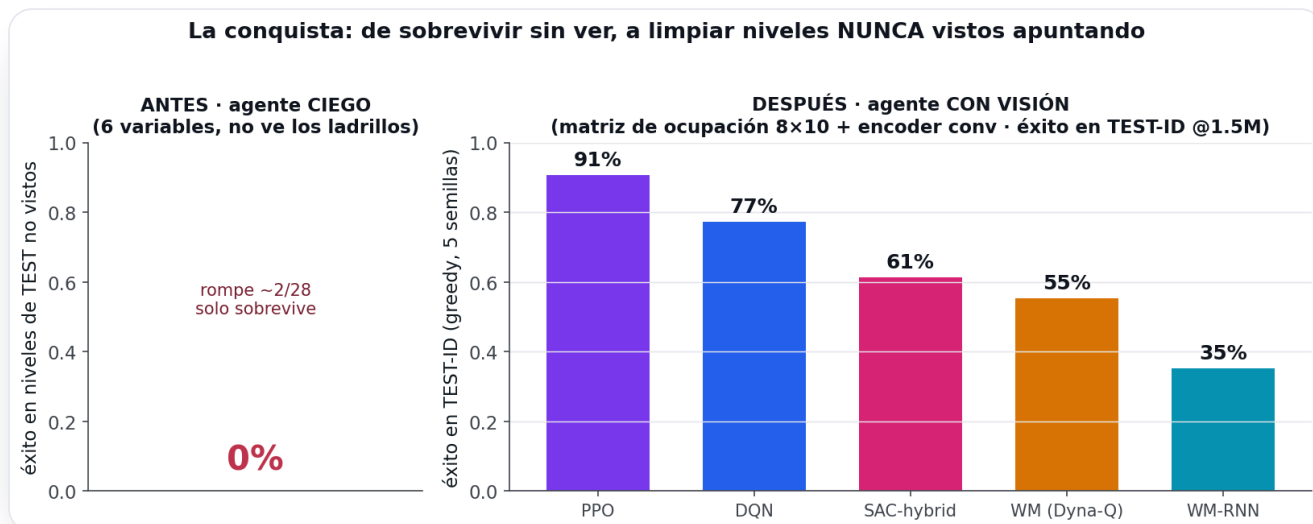
Quédate con esto

Mismo algoritmo, tarea bien planteada: reloj justo + meta limpia + **ojos** (matriz de ocupación) + encoder conv + currículo sobre niveles nuevos. Con esto, como vamos a ver, **PPO funcionó a la primera**.

Ahora limpian niveles que nunca vieron

Lo conseguido, con datos honestos: distribuciones multi-semilla, no el mejor run.

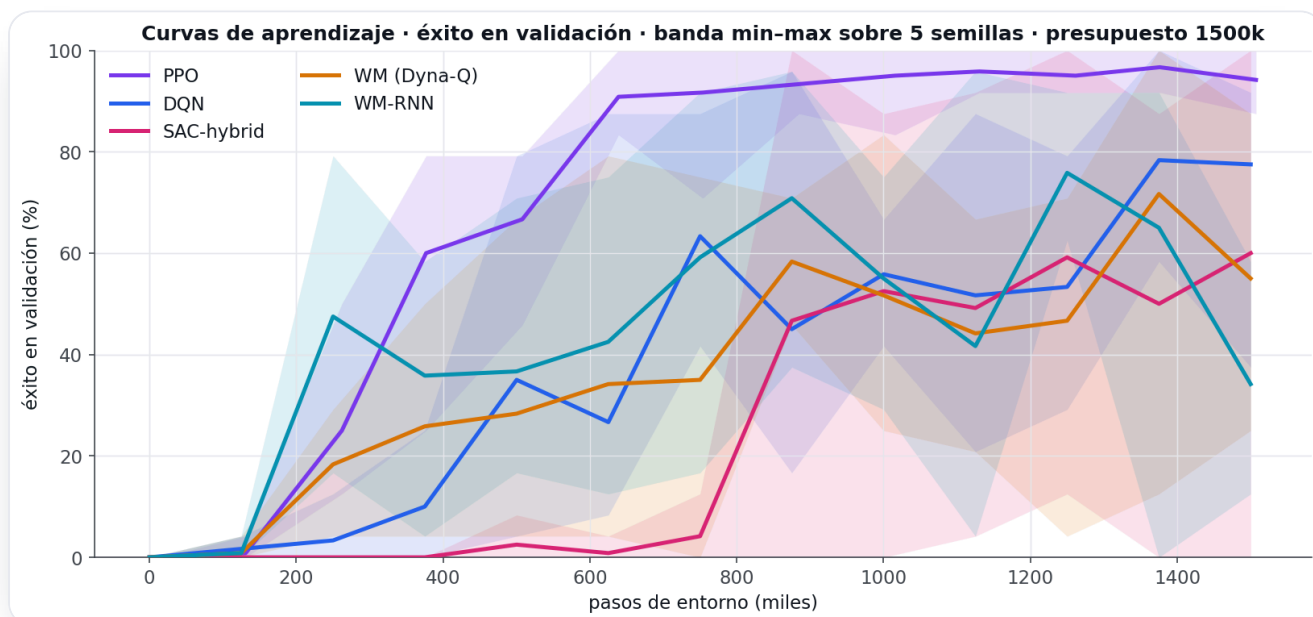
Qué conseguimos y qué significa. Con la tarea bien planteada y el protocolo congelado, varios agentes pasan de no limpiar nada a **resolver la mayoría de niveles no vistos**. Esta es la imagen que lo resume:



F1 · La conquista. Antes (agente ciego): 0% en niveles dispersos. Después (visión + conv, TEST-ID @1.5M, media de 5 semillas): PPO 91%, DQN 77%, SAC-hybrid 61%, WM 55%, WM-RNN 35%.

Cómo aprenden: curvas con banda multi-semilla

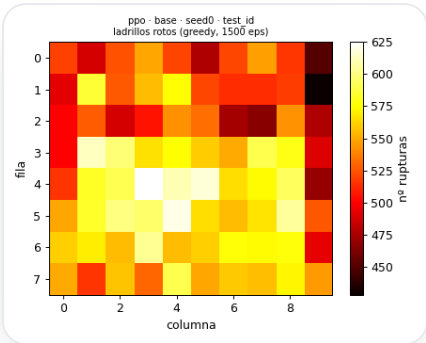
No reportamos «el mejor intento»: reportamos la **distribución** sobre 5 semillas. La línea es la media; la banda, el rango (mín-máx). Una banda ancha = resultado **frágil** (depende de la suerte de la semilla).



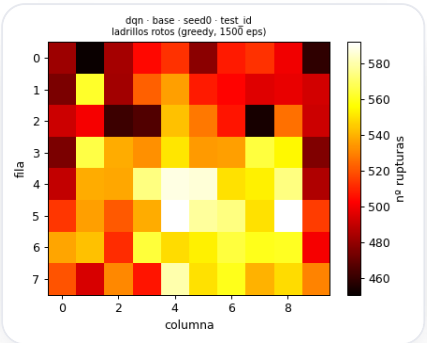
F2 · Curvas de aprendizaje (éxito en validación durante el entrenamiento, banda mín-máx sobre 5 semillas, presupuesto 1,5 M de pasos). PPO sube rápido y alto; DQN converge; SAC tiene banda ancha (frágil); los World Models se quedan más abajo.

La prueba de que *apunta* : mapas de calor

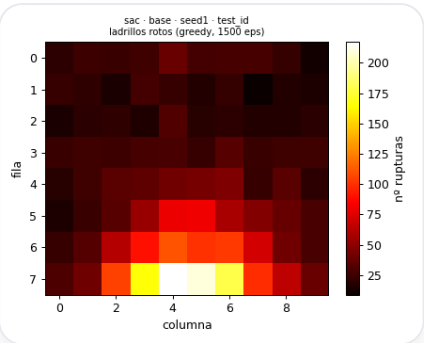
Si el agente ha aprendido a apuntar, los impactos se **concentran** donde hay ladrillos, siguiendo el patrón del nivel. Si no, se reparten al azar (o casi no hay, porque pierde la bola pronto). Compara:



PPO — impactos concentrados: apunta y limpia.



DQN — también apunta, algo menos pulido.



Run colapsada — apenas rompe: pierde la bola enseguida.

F3 · Mapas de calor de ladrillos rotos. La visión convierte el rebote ciego en puntería: los impactos siguen el patrón de cada nivel.

Tabla comparativa final (T1)

Una fila por modelo y por **variante honesta** (SAC-pure y SAC-critic-hybrid **separados**). Métricas en TEST, jugando a lo seguro, media de 5 semillas, presupuesto 1,5 M. En verde, la mejor de cada columna.

MODELO / VARIANTE	TEST-ID	OOD-PATRÓN	OOD-DIFICULTAD	60–80 LADR.	COLAPSOS	%SEM>80	PASOS/LIMPIAR	COMENTARIO
PPO	91%	89%	86%	85%	0%	100%	1923	El mejor y el más fiable.
SAC-pure (actor)	87%	84%	68%	62%	0%	80%	2732	El actor SÍ funciona (ver §6).
DQN	77%	74%	64%	65%	0%	80%	2125	Necesita presupuesto; estable.
SAC-critic-hybrid	61%	60%	51%	54%	20%	40%	1716	Bimodal: a veces colapsa.
WM (Dyna-Q cinemático)	55%	53%	42%	44%	0%	0%	1977	Techo ~55% (ver §6).
WM-RNN (Dyna-Q cinemático)	35%	29%	27%	22%	0%	0%	1370	No-monótono; no mejora con LSTM.

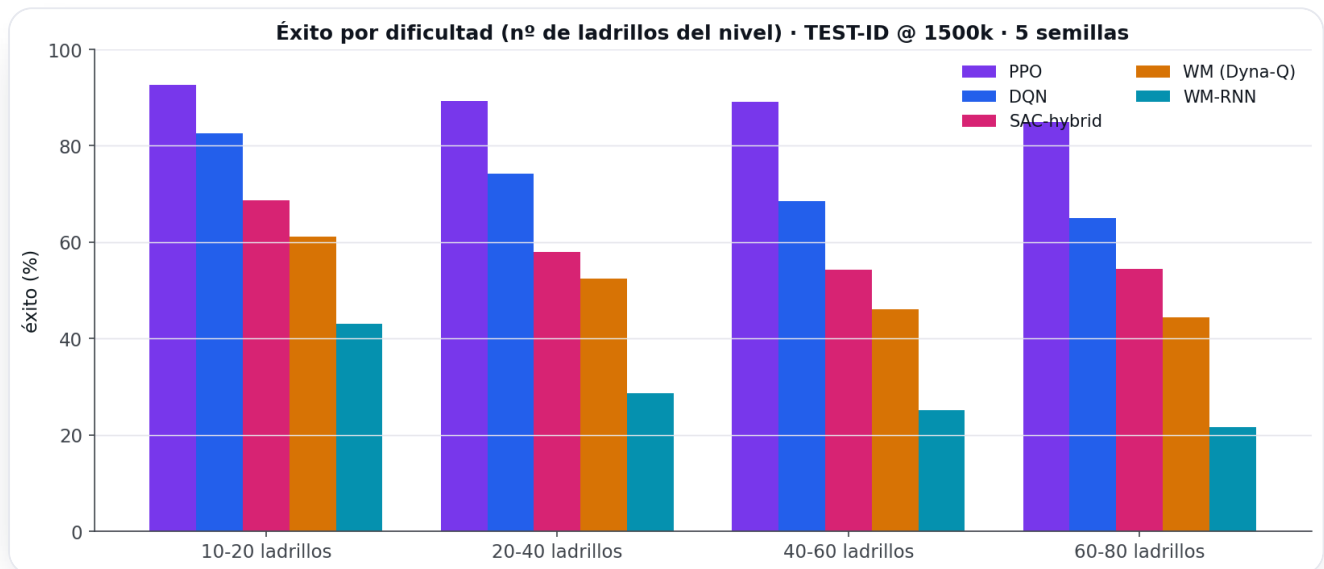
«Colapsos» = % de semillas por debajo del 10% de éxito. «%sem>80» = % de semillas que superan el 80%. Todas las cifras salen de `results/ledger.csv` (160 runs reales).

El presupuesto importa (y revela quién es frágil)

El mismo experimento a tres presupuestos (700 k / 1,5 M / 3 M de pasos) cuenta quién aprende rápido y quién necesita tiempo. A 3 M, los tres model-free convergen arriba; los world-models se estancan.

MODELO	700K	1,5M	3M	LECTURA
PPO	90%	91%	87%	Aprende ya a 700k; meseta. El más sample-efficient.
SAC-pure	25%	87%	91%	Despega entre 700k y 1,5M; arriba a 3M.
SAC-hybrid	4%	61%	88%	Dependencia brutal del presupuesto.
DQN	67%	77%	85%	Mejora monótona; se estabiliza con presupuesto.
WM (Dyna-Q)	33%	55%	56%	Techo ~55%.
WM-RNN	54%	35%	40%	No-monótono (ruido de semilla + alta varianza).

No solo niveles fáciles (F4)



F4 · Éxito por dificultad. Éxito por tramo de nº de ladrillos (TEST-ID @1,5M). PPO mantiene ~85% incluso en los niveles más densos (60–80 ladrillos); los demás caen con la densidad.

Qué ingrediente desbloqueó el aprendizaje (T2 · ablación)

Partimos de la receta completa (DQN, 77%) y quitamos **un** ingrediente cada vez. La caída mide su importancia. Hay dos sorpresas honestas.

INGREDIENTE QUE QUITAMOS	ÉXITO	Δ VS RECETA	COLAPSOS	LECTURA
— (receta completa)	77%	—	0%	Referencia.
Escala 1.0 → 0.25 (de los ladrillos)	1%	-76,5	100%	El ingrediente crítico. Atenuar la señal de ladrillos la mata.
Timeout constante → proporcional	54%	-23,5	0%	El reloj importa, también en sentido inverso.
Conv → lista plana (MLP)	57%	-20,4	0%	El sesgo espacial vale ~20 puntos.
ϵ -decay rápido → lento	75%	-2,6	0%	Poco efecto a este presupuesto.
Sin currículo	75%	-1,9	0%	Casi neutro a 1,5M (ayuda más a bajo presupuesto).
Sin shaping (Φ)	85%	+8,0	0%	¡Quitarlo MEJORA! Confirma que Φ era el saboteador.

Inversión histórica. En la era antigua (rejilla pequeña, lista plana) la escala 0.25 fue **la pieza que desbloqueó** el aprendizaje (atenuaba los ladrillos para no ahogar a la bola). Con el encoder **conv** —que ya separa ladrillos de cinemática— la escala 0.25 lo **mata**. El ingrediente óptimo **depende de la arquitectura**: una lección que solo se ve midiendo, no suponiendo.

Lo conseguido, en una frase

De 0% (ciego) a **91% en niveles no vistos** (PPO), con 0 colapsos y todas las semillas por encima del 80%. Y la ablación confirma la tesis: lo que desbloqueó el problema fueron **ingredientes de formulación** (escala, reloj, conv), no el algoritmo.

Los cinco algoritmos, en breve

Cada uno con su carácter y cómo le fue — al servicio de la comparativa.

Ahora sí, los protagonistas — pero como **reparto**, no como plato principal. La estrella es el resultado; ellos lo explican.

● PPO

GANADOR

MODEL-FREE · ACTOR-CRÍTICO · ON-POLICY

Ajusta directamente la **política** (qué acción tomar) con pasos pequeños y seguros (un «clip» que evita cambios bruscos). Aquí fue el **mejor y el más fiable**: 91% a 1,5M, 0 colapsos, 100% de semillas >80%. Funcionó **a la primera** en cuanto la tarea estuvo bien planteada.

● DQN

MODEL-FREE · BASADO EN VALOR · OFF-POLICY

Aprende el **valor** de cada acción (Q) y elige la mejor. Sólido pero más **hambriento de presupuesto**: 67%→77%→85% conforme entrena más. Estable (0 colapsos), con alguna semilla rezagada a 1,5M.

♥ SAC

HONESTO: 2 VARIANTES

MODEL-FREE · ACTOR-CRÍTICO · OFF-POLICY · ENTROPÍA

Tiene un **actor** (la política) y un **crítico** (el valor). Aquí aparece el hallazgo que **invierte la creencia previa**: ver caja inferior. Honestidad: lo reportamos como dos modelos separados.

● World Model + ♦ WM-RNN

MODEL-BASED · DYNA-Q CON MODELO CINEMÁTICO

Aprenden un **modelo de la dinámica** e «imaginan» pasos para entrenar más rápido. Importante (honestidad): su modelo es **solo cinemático** (predice la bola; los ladrillos quedan fijos en la imaginación). **No** es un simulador del juego. Topan ~55% (MLP) / ~40% (RNN).

! SAC: el actor «de libro» no colapsaba (corrección honesta)

✗ Lo que se creía

«El actor discreto de SAC colapsa, así que usamos la política del **crítico** como apaño» (SAC-critic-hybrid). Era una **sobre-corrección**.

✓ Lo que dicen los datos

El **SAC-pure** (usar el actor para jugar y evaluar) **igual o supera** al híbrido en los tres presupuestos (87% vs 61% a 1,5M) y **colapsa menos** (0% vs 20% de semillas). La premisa no se sostiene bajo protocolo congelado y multi-semilla.

► PARA CURIOSOS · POR QUÉ PASÓ

Con una sola semilla, el actor de SAC sí parecía colapsar — y de ahí salió el apaño del crítico. Pero al medir con **5 semillas** y la receta congelada, el actor resulta competitivo y más estable. Es el peligro de concluir con $n=1$: lo que parece una verdad del algoritmo era ruido de semilla.

World Models: el hallazgo negativo (caracterizado, no escondido)

Los modelos basados en modelo se quedan en ~55% (MLP) y ~40% (RNN), muy por debajo de los model-free. La hipótesis natural era «el LSTM predice peor la dinámica y por eso lastra». **La medimos y la refutamos**: el error de predicción 1-paso del LSTM es prácticamente igual al del MLP ($\times 1,02$). El techo no lo pone el modelo, sino la **dinámica solo-cinemática** (los ladrillos congelados en la imaginación). La recurrencia no aporta y solo añade varianza. Es un **hallazgo negativo principista**: el model-based, así planteado, no ayuda — y sabemos por qué.

PPO: ¿de qué falla ese ~10% restante?

Hasta el mejor, PPO, falla ~1 de cada 10. ¿Es resoluble o es un techo? Lo diagnosticamos: los niveles que falla son **100% limpiables** (el oráculo los limpia), el fallo está **repartido** entre familias (ninguna es un punto ciego), domina «pierde la bola» dejando el 71–80% sin romper (falla **pronto**), y **no mejora** con más presupuesto. Conclusión honesta: es un **techo de control fino** de la política, no de datos ni de visión.

Lo que de verdad nos llevamos

Más allá del Arkanoid: cómo pensar y cómo medir en Deep RL.

✓ Formulación > algoritmo

- El cuello de botella era la **tarea**, no el método.
- Reloj, recompensa, observación, currículo y evaluación importan tanto o más que elegir PPO vs DQN.
- Bien planteado, PPO funcionó a la primera.

✏ La trampa de la métrica

- Recompensa $\neq$ éxito. El ciego acumulaba premio y pasaba 0 niveles.
- El shaping Φ optimizaba la pista, no la meta.
- Medir el **logro real** (éxito en test) cambia las conclusiones.

🔬 Medir con honestidad

- **Multi-semilla**, no el mejor run: salvó 3 conclusiones falsas (SAC «roto», LSTM «peor», currículo «-27»).
- Protocolo **congelado**: sin tocar nada entre comparaciones.
- Cada cifra trazable a un artefacto.

▮ Tres conclusiones que solo se ven midiendo

Model-based	No siempre ayuda. Un modelo de dinámica incompleto (solo cinemático) pone un techo; imaginar sobre él no sube ese techo (y la recurrencia ni ayuda).
SAC frágil	Su fragilidad era en parte nuestra (un apaño de $n=1$). El actor puro, bien medido, es competitivo. La fragilidad real es la alta varianza entre semillas.
El contexto manda	La escala de entrada que desbloqueó la era antigua mata en la nueva. No hay recetas universales; hay recetas para una arquitectura .

La frase que este trabajo puede defender

El problema no era el algoritmo ni el tiempo; era **cómo planteábamos la tarea**. Al arreglar la observación, la recompensa, el currículo y la evaluación, varios algoritmos aprendieron a pasar niveles no vistos — y aquí está cada artefacto que lo demuestra.

Cómo se midió (y por qué te puedes fiar)

Protocolo congelado, infraestructura y reproducibilidad.

El protocolo congelado

Antes de medir, se **congelaron** (y se sellaron con un hash) el generador de niveles, los tres conjuntos de test, las semillas, los presupuestos, la receta base y la forma de evaluar. Una vez congelado, **no se cambia nada** entre comparaciones; cualquier cambio obligaría a re-ejecutar todo. Hash del protocolo: a1ab7ce18d7bad6b.

DECISIÓN	VALOR CONGELADO
Semillas	5 (0–4); se reporta la distribución, no el mejor run.
Presupuestos	700k / 1,5M / 3M pasos de entorno.
Evaluación	greedy (a lo seguro), sobre listas fijas de niveles no vistos.
Test sets	TEST-ID 500 · OOD-patrón 250 · OOD-dificultad 199 (todos limpiables, verificado).
Umbral colapso	una semilla «colapsa» si baja del 10% de éxito.
Ledger	cada run añade una fila a <code>results/ledger.csv</code> (160 runs). Ninguna cifra del documento existe fuera del ledger.

Infraestructura: exprimir la máquina sin tocar la ciencia

El entrenamiento corre en PyTorch sobre la GPU Metal (MPS) de un Apple M3 Ultra. El entorno del juego va en numpy (CPU) y la red en GPU, así que **una sola partida infrautiliza la máquina**. La solución honesta: correr muchas runs **independientes** a la vez (cada una con su configuración congelada, bit a bit idéntica a correrla sola). Medimos el punto óptimo en **24 runs concurrentes** (~5x de throughput; la GPU es el cuello). Resultado: las 75 runs de la comparativa en **64 minutos** en vez de ~3 horas — sin alterar ningún número.

► PARA CURIOSOS · PARALELISMO HONESTO

La clave es que el paralelismo es **entre** runs, no **dentro** de una run. No se cambia el tamaño de lote ni el n° de entornos (eso alteraría los resultados). Se verificó que una run en paralelo da el mismo número que en solitario (84,9% == 84,93%). La escritura del registro es segura entre procesos (lock).

Cómo reproducirlo

```
uv venv --python 3.13 .venv && uv pip install -r requirements.txt
.venv/bin/python gpu/tanda_par.py # comparativa C6 (75 runs, K=24)
.venv/bin/python gpu/report_c6.py # tabla final
```

.venv/bin/python gpu/c1_ppo_failures.py · c2_representation · c3_ablation · c4_sac · c5_world_models

Artefactos · results/ledger.csv (160 runs) · tabla_c6 · ppo_failures · dqn_representation · ablation.csv · sac_variants · wm_variants · hallazgos.md · mapas de calor.

GLOSARIO

Términos, como referencia

Agente / Política (π) — el que decide; la función estado→acción que queremos aprender.

Observación / Estado — lo que el agente «ve». Aquí: bola+pala (6 n°) + matriz de ocupación 8×10.

Recompensa / Shaping — la señal numérica. Shaping = recompensa auxiliar; mal hecha (Φ), sabotadora.

Episodio / Timeout — una partida; su límite de pasos. Si es muy corto, ganar es imposible.

Éxito en test — % de niveles **no vistos** limpiados por completo, jugando a lo seguro. La métrica reina.

Convolución — encoder ligero que lee la rejilla de ladrillos aprovechando su estructura espacial. No es visión por píxeles.

Currículo — entrenar de fácil a difícil, sobre niveles generados, para aprender una política general.

Model-free / Model-based — sin/ con modelo de la dinámica del entorno. Aquí el model-based usa un modelo solo cinemático.

Multi-semilla / Colapso — repetir con varias semillas y reportar la distribución; «colapso» = semilla por debajo del 10%.

Greedy — jugar «a lo seguro», eligiendo siempre la mejor acción conocida (sin explorar). Cómo se evalúa.